

Teil 1 Redis

Die Aufgaben können direkt in der Redis CLI (Aufruf mit `redis-cli`) ausgeführt werden

Aufgaben

A) Strings (Einfache Schlüssel-Wert-Paare)

1. **String erstellen (Create):** Speichere den Namen "Max Mustermann" unter dem Schlüssel `benutzer:name`.
2. **String lesen (Read):** Lasse dir den Wert ausgeben, der unter dem Schlüssel `benutzer:name` gespeichert ist.
3. **String existenz prüfen:** Prüfe, ob der Schlüssel `benutzer:name` überhaupt existiert (ohne den Wert direkt zu lesen).
4. **String überschreiben (Update):** Ändere den Wert von `benutzer:name` auf "Erika Musterfrau".
5. **Numerischen String erhöhen (Update/Counter):** Setze einen neuen Schlüssel `besucher:anzahl` auf den Wert 0. Erhöhe diesen Wert anschließend direkt um 1 (Inkrement).
6. **String löschen (Delete):** Lösche den Schlüssel `benutzer:name` komplett aus der Datenbank.

B) Hashes (Objekte mit Feldern)

7. **Hash erstellen (Create):** Erstelle einen Hash unter dem Schlüssel `produkt:100`. Füge dabei gleichzeitig die Felder `name` mit dem Wert "Laptop" und `preis` mit dem Wert "999" hinzu.
8. **Einzelnes Hash-Feld lesen (Read):** Lasse dir nur den `preis` des Schlüssels `produkt:100` anzeigen.
9. **Alle Hash-Felder lesen (Read):** Lasse dir alle Felder und Werte (Keys und Values) des Hashes `produkt:100` anzeigen.
10. **Hash-Feld hinzufügen/ändern (Update):** Füge dem bestehenden Hash `produkt:100` ein neues Feld `lagerbestand` mit dem Wert "50" hinzu.
11. **Hash-Feld löschen (Delete Field):** Lösche nur das Feld `preis` aus dem Hash `produkt:100`. (Der Rest des Hashes soll bleiben).
12. **Hash mit Ablaufzeit (TTL):** Setze für den gesamten Hash `produkt:100` eine Ablaufzeit (Expiration) von 60 Sekunden.

Lösungen

Aufgabe 1 (Erstellen): Befehl: `SET benutzer:name "Max Mustermann"`

Aufgabe 2 (Lesen): Befehl: `GET benutzer:name`

Aufgabe 3 (Prüfen): Befehl: `EXISTS benutzer:name`

Aufgabe 4 (Überschreiben): Befehl: `SET benutzer:name "Erika Musterfrau"`

Aufgabe 5 (Zähler erhöhen): Befehl 1: `SET besucher:anzahl 0` Befehl 2: `INCR besucher:anzahl`

Aufgabe 6 (Löschen): Befehl: `DEL benutzer:name`

Teil B: Hashes (Objekte)

Aufgabe 7 (Erstellen): Befehl: `HSET produkt:100 name "Laptop" preis "999"`

Aufgabe 8 (Ein Feld lesen): Befehl: `HGET produkt:100 preis`

Aufgabe 9 (Alles lesen): Befehl: `HGETALL produkt:100`

Aufgabe 10 (Feld hinzufügen): Befehl: `HSET produkt:100 lagerbestand "50"`

Aufgabe 11 (Feld löschen): Befehl: `HDEL produkt:100 preis`

Aufgabe 12 (Ablaufzeit setzen): Befehl: `EXPIRE produkt:100 60`

Teil 2 MongoDB

Die Aufgaben können direkt in der MongoDB Shell (mongosh) ausgeführt werden

Es werden zwei "Collections" (Sammlungen): `benutzer` und `produkte` genutzt. Diese Collections müssen vorher nicht angelegt werden. Anders als bei SQL-Datenbanken (wie PostgreSQL), wo man zwingend erst `CREATE TABLE` ausführen muss, arbeitet MongoDB „dynamisch“. Sobald ein Befehl um Daten zu speichern ausgeführt wird (z. B. `insertOne`), prüft MongoDB im Hintergrund: Gibt es diese Collection (`benutzer`) schon? Wenn ja: Füge das Dokument hinzu. Wenn nein: Erstelle die Collection sofort automatisch und füge dann das Dokument hinzu.

Aufgaben

A) Einfache Dokumente (Benutzer)

1. Dokument erstellen (Create):

Füge in die Collection `benutzer` ein Dokument ein, das nur das Feld `name` mit dem Wert "Max Mustermann" enthält.

2. Dokument lesen (Read):

Suche in der Collection `benutzer` nach dem Dokument, wo der `name` "Max Mustermann" ist.

3. Existenz prüfen (Count):

Zähle, wie viele Dokumente es in `benutzer` gibt, die den Namen "Max Mustermann" haben (um zu prüfen, ob es existiert).

4. Feld ändern (Update - \$set):

Ändere bei dem Benutzer "Max Mustermann" den Namen auf "Erika Musterfrau". (Wichtig: Nutze den Operator `$set`).

5. Wert erhöhen (Update - \$inc):

Erstelle erst ein neues Dokument in einer neuen Collection `statistik` mit `{ art: "besucher", anzahl: 0 }`.

Erhöhe danach den Wert `anzahl` dieses Dokuments um 1.

6. Dokument löschen (Delete):

Lösche den Benutzer mit dem Namen "Erika Musterfrau" komplett aus der Collection.

B) Komplexe Dokumente & Felder (Produkte)

7. Dokument mit ID erstellen (Create):

Füge ein Produkt in die Collection `produkte` ein. Setze die `_id` manuell auf 100. Das Dokument soll außerdem `name: "Laptop"` und `preis: 999` enthalten.

8. Nur ein bestimmtes Feld lesen (Projection):

Lasse dir vom Produkt mit der `_id: 100` nur den `preis` anzeigen (blende die ID dabei explizit aus).

9. Ganzes Dokument lesen (Read):

Lasse dir das komplette Dokument des Produkts mit der `_id: 100` anzeigen.

10. Neues Feld hinzufügen (Update - \$set):

Füge dem Produkt 100 ein neues Feld `lagerbestand` mit dem Wert 50 hinzu.

11. Feld entfernen (Update - \$unset):

Lösche nur das Feld `preis` aus dem Dokument des Produkts 100 (das Feld `name` und `lagerbestand` sollen bleiben).

12. Alles anzeigen:

Lasse dir zur Kontrolle alle Dokumente anzeigen, die sich aktuell in der Collection `produkte` befinden.

Lösungen

Aufgabe 1 (Erstellen)

```
db.benutzer.insertOne({ name: "Max Mustermann" })
```

Aufgabe 2 (Lesen)

```
db.benutzer.find({ name: "Max Mustermann" })
```

Aufgabe 3 (Prüfen/Zählen)

```
db.benutzer.countDocuments({ name: "Max Mustermann" })
```

Aufgabe 4 (Ändern)

```
db.benutzer.updateOne({ name: "Max Mustermann" }, { $set: { name: "Erika  
Musterfrau" } })
```

Aufgabe 5 (Inkrementieren)

```
Schritt 1: db.statistik.insertOne({ art: "besucher", anzahl: 0 })
```

```
Schritt 2: db.statistik.updateOne({ art: "besucher" }, { $inc: { anzahl: 1 } })
```

Aufgabe 6 (Löschen)

```
db.benutzer.deleteOne({ name: "Erika Musterfrau" })
```

Aufgabe 7 (Erstellen mit ID)

```
db.produkte.insertOne({ _id: 100, name: "Laptop", preis: 999 })
```

Aufgabe 8 (Nur Preis lesen - Projektion)

```
db.produkte.findOne({ _id: 100 }, { preis: 1, _id: 0 })
```

(Erklärung: 1 bedeutet anzeigen, 0 bedeutet ausblenden)

Aufgabe 9 (Alles lesen)

```
db.produkte.findOne({ _id: 100 })
```

Aufgabe 10 (Feld hinzufügen)

```
db.produkte.updateOne({ _id: 100 }, { $set: { lagerbestand: 50 } })
```

Aufgabe 11 (Feld entfernen)

```
db.produkte.updateOne({ _id: 100 }, { $unset: { preis: "" } })
```

Aufgabe 12 (Alle Produkte listen)

```
db.produkte.find()
```
